

13 — Database Transactions & Atomic Swap

What it is

A **database transaction** is a way to bundle several database operations — say, a `DELETE` followed by a handful of `INSERT`s — into one all-or-nothing unit. Either **every** operation in the transaction succeeds and gets permanently saved (a **COMMIT**), or if anything goes wrong partway through, **everything** done so far in that transaction gets undone as if it had never happened (a **ROLLBACK**). There is no in-between state where some of the operations took effect and others didn't — the database guarantees you land on one side of that line or the other, never straddling it.

Atomic swap is the specific pattern this project (R11) uses transactions for: whenever a category's or a company's search rows need updating, the old rows are deleted and the new rows are inserted inside one single transaction. "Atomic" here means "indivisible" — from the outside, the update either hasn't happened at all yet (old rows still there) or has fully happened (new rows in place). A crash in the middle can never leave that category or company with a half-deleted or mixed set of rows.

Real-life analogy

Think of swapping the tyres on a car. A mechanic doesn't take one wheel off, leave the car on a single wheel overnight "just in case," and put the new wheel on tomorrow. They jack the car up, and the whole swap — old wheel off, new wheel on, bolts tightened — happens as one uninterrupted operation before the car ever touches the ground again. If the mechanic got a phone call and had to walk away with one wheel off and the replacement not yet bolted on, the car simply isn't driven until the job finishes — it's never rolled out onto the road in that half-finished state.

A database transaction is the jack stand. It holds the "car" (the row of data) suspended in a safe, uncommitted state while the swap happens, and only lowers it back onto the road (commits) once every step of the swap is done. If something interrupts the mechanic partway through, the transaction's rollback is like automatically putting the original wheel straight back on — nobody driving past ever sees the car missing a wheel.

Why we used it here

Without a transaction, a naive update would be two separate, independently-committed steps: `DELETE` the old rows for this category/company, and then, as a second and completely separate step, `INSERT` the new rows. Each step commits on its own the instant it runs.

Now imagine the ETL process crashes in the gap between those two steps — a server restart, a dropped network connection to Postgres, an out-of-memory kill, anything. The `DELETE` already committed. The `INSERT` never ran. That category or company now has **zero rows** in `category_vectors` or `company_vectors` — completely invisible to search — and stays that way until someone notices and re-runs the ETL. For a live B2B marketplace search feature, that's not a cosmetic glitch; it's a real availability bug. A buyer searching for that category or company simply gets no results, with no error anywhere to explain why.

Wrapping the `DELETE` and all the `INSERTs` in one transaction closes that gap entirely. If the crash happens anywhere inside the transaction — after the `DELETE`, after one `INSERT`, after five `INSERTs`, it doesn't matter — Postgres rolls the whole thing back and the original rows are exactly as they were before the update was attempted. The category/company is never search-invisible; worst case, it's still serving the *previous* version of its data until the next successful ETL run.

Code walkthrough

The with conn: pattern

From `bilingual_etl/load/pgvector_client.py`:

```
def upsert_category_vectors(conn, category_id: str, rows: list[dict]) -> None:
    with conn:
        with conn.cursor() as cur:
            cur.execute("DELETE FROM category_vectors WHERE category_id = %s", (category_id,))
            for row in rows:
                bm25_hu = row["bm25_tokens"] if row["language"] == "hu" else None
                bm25_en = row["bm25_tokens"] if row["language"] == "en" else None
                cur.execute(
                    """
                    INSERT INTO category_vectors
                        (category_id, language, narrative, embedding,
                         bm25_tokens_hu, bm25_tokens_en, metadata, content_hash)
                    VALUES (%s, %s, %s, %s::vector,
                             to_tsvector('simple', %s), to_tsvector('simple', %s), %s, %s)
                    """,
                    (
                        category_id,
                        row["language"],
                        row["narrative"],
                        _format_vector(row["embedding"]),
                        bm25_hu,
                        bm25_en,
                        Json(row.get("metadata", {})),
                        row["content_hash"],
                    ),
                )
            logger.info(f"Upserted {len(rows)} category_vectors row(s) for category_id={category_id}")
```

`upsert_company_vectors()` in the same file follows the identical shape — `DELETE` for that `company_id`, then a loop of `INSERTs` — all inside its own `with conn:` block.

The important line is `with conn:`. In `psycopg2`, the connection object itself is a Python **context manager**. Using it with `with` means:

- If the code inside the block runs to the end **without raising an exception**, `psycopg2` automatically calls `conn.commit()` for you.
- If **any** exception is raised anywhere inside the block — a bad SQL statement, a constraint violation, a network drop, a process kill that Python happens to catch, literally anything — `psycopg2` automatically calls `conn.rollback()` instead, and then re-raises the exception so the caller still finds out something went wrong.

The programmer never writes `.commit()` or `.rollback()` by hand in either function above. That matters because it removes an entire category of bugs: the programmer doesn't have to anticipate every possible failure path and remember to roll back on each one. A crash the developer never imagined — a code path nobody wrote a test for — still gets rolled back correctly, because the safety net is structural (built into the `with` statement), not something that depends on the programmer remembering to add a `try/except/rollback` around every possible failure.

The `to_tsvector('simple', NULL)` detail

Each row only ever fills in **one** of `bm25_tokens_hu` / `bm25_tokens_en`, matching its own `language` — the other column is left `NULL`:

```
bm25_hu = row["bm25_tokens"] if row["language"] == "hu" else None
bm25_en = row["bm25_tokens"] if row["language"] == "en" else None
```

Both values are passed into the SQL as parameters to `to_tsvector('simple', %s)` regardless of which one is `None`. This works cleanly because Postgres's `to_tsvector()` function returns `NULL` when given a `NULL` input — it doesn't error and doesn't need a special case. So a Hungarian row ends up with a real `tsvector` in `bm25_tokens_hu` and a plain `NULL` in `bm25_tokens_en`, with no `if/else` branch needed in the SQL itself to skip the "wrong" language's column. Python's `None` passed as a bound parameter becomes SQL `NULL`, and `to_tsvector('simple', NULL)` just quietly becomes `NULL` too.

The `::vector` casting detail

```
def _format_vector(embedding: list[float]) -> str:
    return "[" + ",".join(str(x) for x in embedding) + "]"
```

Postgres's `vector` type (from the `pgvector` extension) doesn't have a native Python type that `psycopg2` understands out of the box — that support normally comes from installing the separate `pgvector` Python package, which this project deliberately chose not to add as a dependency for something with such an easy manual workaround. Instead, `_format_vector()` turns a Python list of floats like `[0.1, 0.2, 0.3]` into the plain string `"[0.1,0.2,0.3]"` — exactly the text format Postgres's `vector` type literal expects. That string is passed as an ordinary SQL string parameter, and the SQL itself does the real conversion with an explicit cast: `%s::vector`. Postgres reads the string and casts it to a native `vector` value at INSERT time. No extra dependency, no custom type adapter — just a string built in Python and a cast written directly in the SQL.

The rollback simulation

`bilingual_etl/scripts/simulate_atomic_rollback.py` proves the whole guarantee actually holds, rather than just asserting it in a comment. Step by step:

1. **Insert 2 baseline rows** for a fake test category, `SIMULATION_TEST_CATEGORY_999999` — one Hungarian narrative row, one English narrative row — simulating "this is what's already live in production."
2. **Start a new transaction that deletes those rows and inserts one new row**, then **deliberately raises a Python exception on purpose** right after the `INSERT`, inside `_attempt_swap_with_simulated_crash()`:

```
def _attempt_swap_with_simulated_crash(conn) -> None:
    with conn:
```

```

with conn.cursor() as cur:
    cur.execute("DELETE FROM category_vectors WHERE category_id = %s", (TEST_CATEGORY_ID,))
    cur.execute(
        """
        INSERT INTO category_vectors
            (category_id, language, narrative, embedding, metadata, content_hash)
        VALUES (%s, %s, %s, %s::vector, %s, %s)
        """,
        (TEST_CATEGORY_ID, "hu", "narrativa amely soha nem kerul commitolasra",
         "[" + ",".join(["0.0"] * 768) + "]", Json({}), "hash_that_never_commits"),
    )
    raise RuntimeError("Simulated crash mid-transaction (e.g. process killed, network drop)")

```

The `raise RuntimeError(...)` stands in for any of the real-world crash scenarios named right in the message — the process getting killed, the network dropping — anything that would abort execution mid-transaction. Because this happens inside the `with conn:` block, psycopg2 rolls the transaction back automatically the instant the exception propagates out.

3. **Check what's actually in the database afterward** — `_fetch_current_rows()` queries `category_vectors` for that same test category ID again.

4. **Confirm the rows are still exactly the original 2 baseline rows** — proving the failed `DELETE+INSERT` never took effect at all.

This is the actual, real logged output from running the script, saved verbatim at `uat-evidence/R11-atomic-swap/rollback_simulation.log`:

```

EVIDENCE_ATOMIC_SWAP_SIMULATION_START
Baseline established: 2 row(s)
EVIDENCE_ATOMIC_SWAP_SIMULATED_FAILURE_CAUGHT: Simulated crash mid-transaction (e.g. process killed, network d
EVIDENCE_ATOMIC_SWAP_ROLLBACK_RESULT: rollback_ok=true (baseline_rows=2, post_crash_rows=2)
EVIDENCE_ATOMIC_SWAP_SIMULATION_DONE

```

The actual proof is `rollback_ok=true` together with `baseline_rows=2` and `post_crash_rows=2` being **equal**. If the transaction had *not* rolled back correctly, `post_crash_rows` would have come out wrong — `0` if only the `DELETE` had somehow persisted on its own, or some other number if the one stray `INSERT` had leaked through without the `DELETE`. Two matching row counts, with the same content, is what "nothing happened" looks like when you check it from the outside.

This log was captured as formal **UAT (User Acceptance Testing) evidence** for the project. `docs/Client_UAT_Evidence_README.md` requires a specific named log marker — `EVIDENCE_ATOMIC_SWAP_ROLLBACK_RESULT` showing `rollback_ok=true` — as proof this safety guarantee actually works, not just a claim in a design doc that it does. That's why the log lives at `uat-evidence/R11-atomic-swap/rollback_simulation.log` as part of the project's formal record.

A real bug found and fixed in this same phase

While wiring up this simulation, it turned out `bilingual_etl/load/db.py` was never actually loading the project's `.env` file — it only read whatever environment variables were *already* set:

```

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://bilingual:bilingual@localhost:5432/bilingual_search",
)

```

If nothing else had loaded `.env` first, any script run directly — like `simulate_atomic_rollback.py` — would silently fall back to that hardcoded default and try to connect with the wrong host/port/credentials, with no obvious error pointing at the real cause. The fix was to call `load_dotenv()` exactly once, in the top-level package's `bilingual_etl/__init__.py`:

```
from dotenv import load_dotenv

load_dotenv()
```

Because Python runs a package's `__init__.py` the first time *any* module inside that package is imported, this guarantees `.env` is loaded before `db.py` (or anything else in the package) ever runs — no matter which script happens to be the entrypoint. The alternative — sprinkling `load_dotenv()` calls across every individual script — would have worked too, but only as long as every future entrypoint remembered to add it. Centralizing it in `__init__.py` removes that "remember to do this every time" burden entirely.

How to explain this in an interview/to a teammate

"We wrap every update to a category's or company's search rows in a single database transaction — a DELETE of the old rows followed by a loop of INSERTs for the new ones, all inside one `with conn:` block with `psycopg2`. That gives us an all-or-nothing guarantee: either the whole swap commits, or if anything throws partway through — a bad row, a dropped connection, a crash — `psycopg2` automatically rolls back everything in that block, no manual `.commit()/rollback()` calls needed on our part. Without that, a naive two-step DELETE-then-INSERT could crash in the gap between the two and leave that company or category with zero search rows until someone noticed and reran the pipeline — a real availability bug for a live search feature, not just a data-quality nitpick. We didn't just claim this works — we wrote a simulation script that inserts baseline rows, starts a transaction that deletes them, inserts a replacement, and then deliberately raises an exception before it can commit, then checks that the original rows are still there afterward. The logged result — `rollback_ok=true` with matching baseline and post-crash row counts — is the actual proof, and it's saved as UAT evidence for the client. Two smaller details worth mentioning if asked: we let `to_tsvector()`'s own NULL-in-NULL-out behavior handle the unused bilingual column instead of writing conditional SQL, and we cast embeddings to Postgres's `vector` type with a plain string literal and `::vector` in the SQL rather than pulling in the separate `pgvector` Python package just for that."